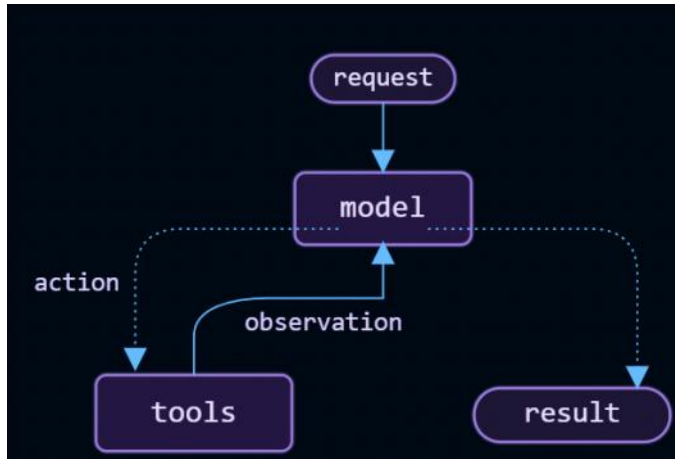


Agents - LangChain

Thursday, October 1, 2026 8:05 AM

Agents: An agent is a model calling tools in a loop until a given task is complete.

An AI agent is smart system that can **understand a goal, make decisions, perform actions, observe the results**, and continue **until the goal is completed** with no human intervention.



Think of it this way:

- A **traditional program** follows fixed instructions.
- An **LLM (like ChatGPT)** generates text based on your prompt.
- An **AI agent** uses an LLM as its "brain" but can also **plan, use tools, remember information, and take actions**.

A Simple Real-Life Analogy: Imagine you want to book a vacation.

Normal Program:

You manually,

1. Search flights
2. Search hotels
3. Compare prices
4. Book everything

The software only does exactly what you click.

LLM: You ask: Suggest a 5-day trip to Goa.

It gives you an itinerary but can't actually book anything by itself.

AI Agent: You tell it: Plan a 5-day Goa trip under \$40,000

The agent:

1. Searches flights
2. Compares prices
3. Finds hotels
4. Check weather
5. Creates an itinerary

6. Asks for confirmation
7. Books everything.

It works toward the goal instead of just answering a question.

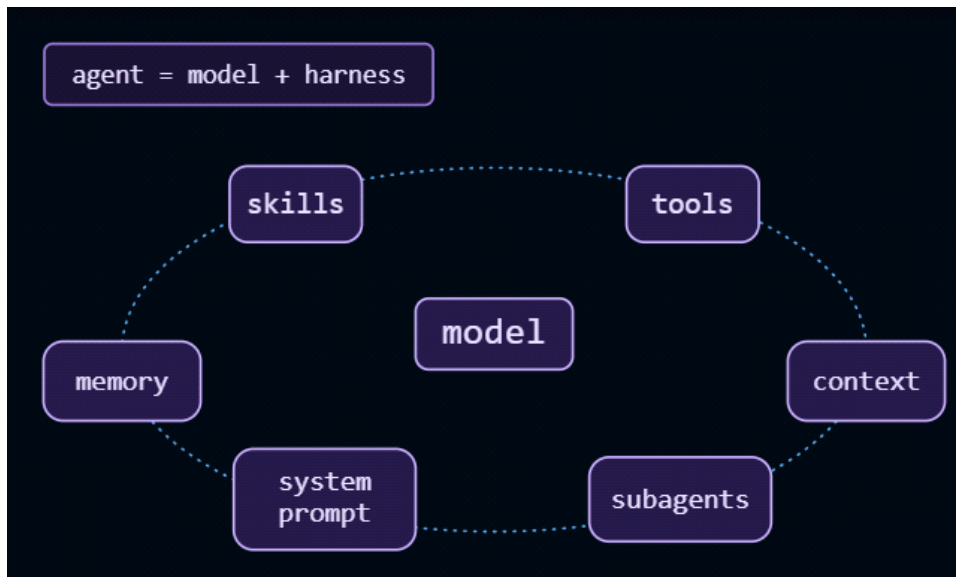
Different Between an LLM and an Agent:

LLM	AI Agent
Answers Questions	Completes goals
Produces text	Takes actions
One prompt -> One response	Multi step workflow
Doesn't plan much	Plans tasks
No built-in memory	Often maintains memory
Usually no external tools	Uses tools, APIs, DB, Browsers etc
Passive	Active

Core components:

This diagram explains the core building blocks of an AI agent.

Agent = Model + Harness



What does "Agent = Model + Harness" Mean?

Think of it like a human:

Model = Brain (LLM): the reasoning engine that understands language and decides what to do next.

Ex: GPT, Claude, Gemini, Mistral, Llama etc.

Analogy: The doctor's medical knowledge. Diseases, Treatments, Diagnosis
But can't diagnose a patient without more information.

Harness = Everything around the brain that helps it perform tasks.

Analogy: Patient History, Blood reports, X-rays, Medical instruments, Hospital staff, Previous records, Specialists etc.

Without these, even an expert doctor can't do the job effectively.

Doctor = Medical Knowledge + Hospital Resources

A brain alone can't send emails, search the internet, or remember previous conversations. Similarly, an LLM alone only predicts the next word.

AI agent = LLM + Tools + Memory, Context + Skills

Ex: memory, tools, instructions, content, specialized skills, other agents to collaborate with together, they form an AI agent.

Memory: Stores information from previous steps or interactions.

Ex: "The user prefers vegetarian food."

Planning: Breaks a complex goal into smaller tasks.

Ex: Goal: Create a company report.

Plan: Read docs, Extract facts, Summarize, Create slides, Email manager.

Tools: Agents become much more capable when they can use tools, such as:

- Search the web
- Read files
- Execute Python code
- Query databases
- Send emails
- Call APIs
- Interact with applications

Environment: The external world the agent interacts with, including files, websites, databases, devices or cloud services.

Example: Tell me today's weather.

LLM: I don't know today's weather unless I have current Information.

AI Agent:

- Uses a weather API.
- Retrieves today's forecast.
- Explains whether you need an umbrella.

Skills: Skills are reusable capabilities the agent has learned or been programmed with.

Examples: Think of skills as a person's professional expertise.

Customer support agent: answer FAQs, summarize complaints, classify tickets

Coding agent: explain code, debug programs, generate tests

Research agent: summarize papers, compare articles, create reports

Tools: Tools allow the agent to interact with the outside world.

Context: Context is the information the model currently uses to make decisions.

It includes: your prompt, uploaded documents, previous conversation, retrieved knowledge (RAG), instructions, task details.

Subagents: Sometimes one agent delegates work to other specialized agents.

Ex: Imagine building a travel plan.

Main Agent: "I need help!"

Flight Agent: Finds flights

Hotel Agent: Books hotels

Weather Agent: Check forecast

Budget Agent: Calculates costs

Main Agent : Combines everything into one plan

This is a multi-agent system, where each subagent focuses on a specific responsibility.

System Prompt: The system prompt defines the agent's role and behavior.

Ex:

You are a helpful medical assistant.

Never guess.

Always explain in simple language.

Cite reliable sources when appropriate.

This acts like the agent's job description and operating rules.

Memory: Memory lets the agent retain information beyond the current prompt.

There are two common types: Short-term-memory, Long-term-memory

Why are AI Agents Popular?

They can automate complex workflow such as:

- Customer support
- Research assistance
- Coding and debugging
- Financial analysis
- Medical document summarization with human oversight
- Personal assistants
- RAG systems
- Business process automation

Agent Frameworks: LangChain, LlamaIndex, Haystack, Microsoft AutoGen, CrewAI

Type Of AI Agent Memory:

1. Short -Term Memory (Conversation Memory)

Stores information from the current session.

Example:

User:

My name is Rahul.

Later:

User:

What's my name?

The agent remembers **Rahul** from the current conversation.

Used for:

- Chatbots
- Customer support
- Multi-turn conversations



2. Long-Term Memory

Stores information across different sessions.

Example:

Monday:

I prefer Python.

Next week:

Recommend a backend framework.

The agent remembers your preference for Python and suggests FastAPI or Django.

Typically stored in:

- Databases
- Vector databases
- Key-value stores

3. Working Memory

3. Working Memory

Stores intermediate information while solving a task.

Example:

User:
Analyze this CSV.

The agent:

- Loads the CSV
- Calculates totals
- Identifies trends
- Generates charts

The intermediate calculations are kept only until the task completes.

4. Episodic Memory

4. Episodic Memory

Stores past experiences or completed tasks.

Example:

Yesterday:
Generated monthly sales report.

Today:

Generate the report again.

The agent remembers the previous workflow and can reuse it or adapt it.

5. Semantic Memory

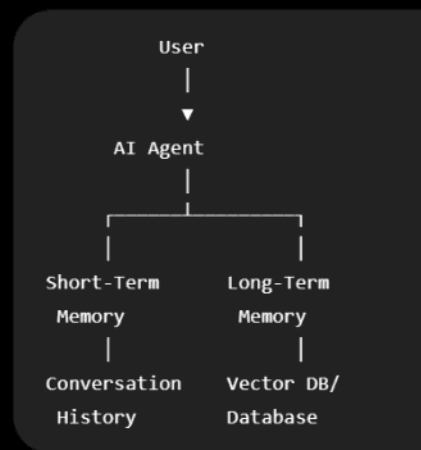
Stores general facts and learned knowledge.

Example:

Company HQ = Bengaluru
CEO = John Smith

Unlike episodic memory, it stores facts rather than events.

Memory Architecture



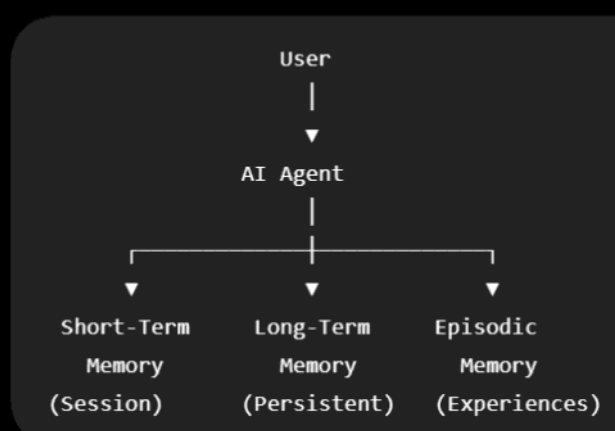
Memory in LangChain

Examples include:

- ConversationBufferMemory
- ConversationSummaryMemory
- ConversationBufferWindowMemory
- VectorStoreRetrieverMemory

Each balances context retention with token usage.

Memory Architecture



Comparison

Memory Type	Stores	Lifetime	Example
Short-term	Current conversation and task context	Minutes to session end	Remembering your name during a chat
Long-term	Persistent facts and user preferences	Days to years	Remembering your preferred programming language
Episodic	Past interactions, workflows, and outcomes	Long-term	Reusing the steps from a previous report generation task

ReAct Loop

Thought
↓
Action
↓
Observation
↓
Thought
↓
Action
↓
Observation
↓
Final Answer

Why ReAct is Better Than a Simple LLM

Simple LLM

Question
|
▼
LLM
|
▼
Answer

- Single step
- No external tools
- May hallucinate if it lacks information

ReAct Agent

Question
|
▼
Reason
|
▼
Tool
|
▼
Observation
|
▼
Reason
|
▼
Final Answer

- Multi-step reasoning
- Uses tools
- Can gather real-time information
- Produces more reliable answers for tasks requiring external data

Common Tools Used

- Web search
- Calculator
- SQL database
- Vector database (RAG)
- Email API
- Calendar API
- Weather API
- Code interpreter

Plan and Execute agent pattern

User:

Plan a 3-day trip to Goa within ₹20,000.

Step 1 – Planner

The planner creates a plan:

1. Find flights
2. Find hotels
3. Estimate local transport
4. Recommend attractions
5. Calculate total budget
6. Generate itinerary

Step 2 – Executor

The executor performs each step:

- Search flight options
- Search hotels
- Estimate transport costs
- Find attractions
- Compute total cost
- Produce the itinerary

Final Output

A complete travel plan with costs and recommendations.

How do you design and define tools for an AI agent?

1. Define a single Responsibility

Each tool should do **one well-defined task**.

✓ Good:

- `get_weather(city)`
- `send_email(to, subject, body)`
- `search_documents(query)`

✗ Bad:

`manage_everything()`

2. User Clear name

Tool names should describe exactly what they do.

Good examples:

`search_customer`
`create_invoice`
`get_stock_price`
`book_meeting`

3. Define Parameter Clearly

Specify required inputs, their types, and constraints.

Example:

Specify required inputs, their types, and constraints.

Example:

```
</> JSON

{
  "name": "get_weather",
  "parameters": {
    "city": "string"
  }
}
```

Another example:

```
</> JSON

{
  "name": "send_email",
  "parameters": {
    "to": "string",
    "subject": "string",
    "body": "string"
  }
}
```



The more precise the schema, the more reliable the tool calls.

4. Write Good Descriptions

Descriptions help the LLM decide when to use a tool.

Example:

Get the current weather for a city.

Use this tool whenever the user asks about
weather, temperature, rainfall,
or forecast.

A detailed description reduces incorrect tool selection.

5. Return Structured Output

Prefer JSON over free text.

Good:

```
</> JSON
{
  "temperature": 24,
  "humidity": 65,
  "condition": "Rain"
}
```

Avoid:

Looks like rain today.

Structured outputs are easier for the LLM to interpret and combine with other tools.

6. Handle Error

Return meaningful errors instead of crashing.

Example:

```
</> JSON
{
  "error": "City not found"
}
```

The agent can then ask the user for clarification or retry.

How do you handle agent failures and implement error recovery?

Types of Agent failures:

- Tool/API failures (timeouts, network issues)
- Invalid function arguments
- Hallucinated tool calls
- Missing or insufficient information
- Rate limits
- Parsing JSON errors
- Infinite reasoning loops
- Low-confidence retrieval in RAG

Error Recovery Strategies:

1. Retry Transient Failures

Example:

- API timeout
- Temporary service unavailable

Best practice:

- Limit retry attempts (e.g., 3)
- Use exponential backoff

Example:

- API timeout
- Temporary service unavailable

Best practice:

- Limit retry attempts (e.g., 3)
- Use exponential backoff

2. Validate Tool Inputs

Validate parameters before calling a tool.

Example:

User:

Book a flight on February 30.

The agent should detect the invalid date and ask for clarification instead of calling the booking API.

3. Handle Tool Errors Gracefully

Tools should return structured errors.

Example:

```
</> JSON
{
  "error": "Customer ID not found"
}
```

The agent can respond:

"I couldn't find that customer. Could you verify the customer ID?"

4. Fallback Strategy

If the preferred tool fails:

- Try another tool.
- Use cached results.
- Ask the user for more information.
- Return a partial answer if appropriate.

Example:

```
Weather API A
|
Failed
|
▼
Weather API B
```

5. Human in the loop

Escalate high-risk tasks to a human.

Escalate high-risk tasks to a human.

Examples:

- Medical advice
- Legal decisions
- Financial transactions
- Large payments

6. Confidence Thresholds

In RAG systems:

- If retrieval confidence is low, don't guess.
- Ask a follow-up question or state that the information isn't available.

Example:

"I couldn't find enough information in the available documents to answer that accurately."

7. Limit Agent Loops

Prevent endless reasoning.

Example:

- Maximum of 5 reasoning iterations.
- Stop if no progress is made.

This avoids infinite loops and unnecessary costs.

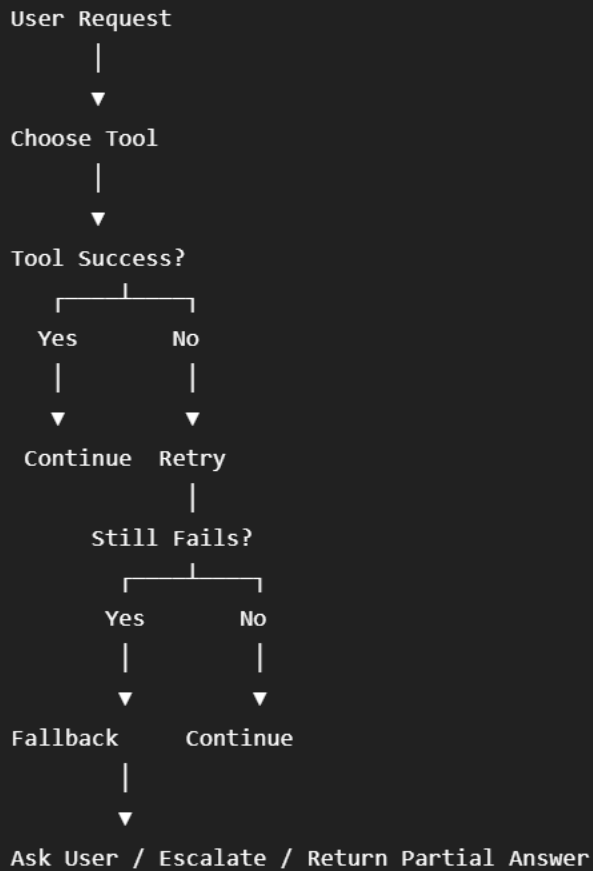
8. Log and Monitor

Capture:

- User request
- Selected tools
- Tool outputs
- Errors
- Latency
- Token usage

This helps debug failures and improve the agent over time.

Error Recovery Workflow

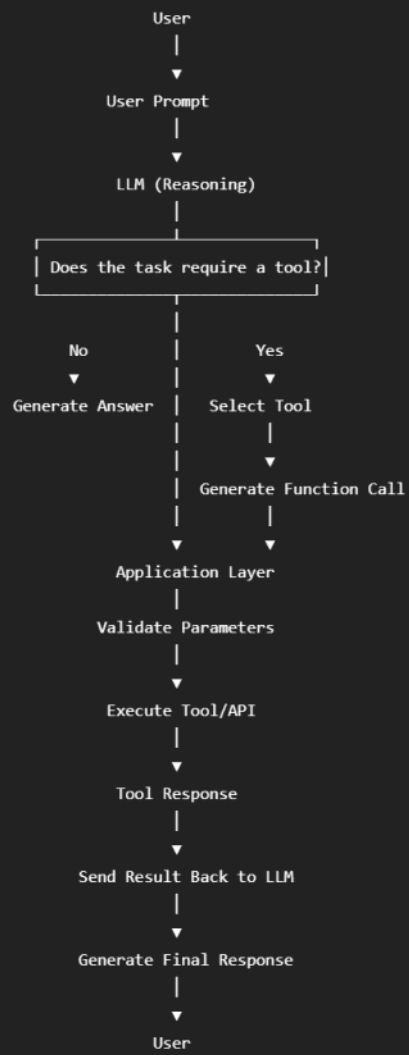


How to Manage KV Cache Memory:

- Limit Context Length
- Sliding Window Attention
- Prompt Compression / Summarization
- Prefix Cache sharing
- Batch Scheduling
- Cache Eviction
- Quantized KV Cache
- Offloading

Explain the tool-calling Architecture in an AI agent? How do you design a tool-calling framework?

Tool Calling Architecture



Components

1. User Prompt

The user asks a question.

Example:

"What's the weather in Bangalore?"

2. LLM Reasoning

The model decides:

- Can I answer from my own knowledge?
- Or do I need an external tool?

3. Tool Selection

The model chooses the appropriate tool.

Examples:

- Weather API
- SQL Database
- Search API
- Calculator
- RAG Retriever

4. Function Call Generation

The model generates structured arguments.

Example:

```
</> JSON  
  
{  
  "function": "get_weather",  
  "arguments": {  
    "city": "Bangalore"  
  }  
}
```

5. Application Layer

Your backend:

- Validates parameters
- Checks authentication
- Executes the API
- Handles retries and errors

The LLM never directly calls APIs.

6. Tool Execution

Example:

```
get_weather("Bangalore")  
  
↓  
  
24°C  
Rain probability 70%
```

7. Observation

The tool result is returned to the LLM.

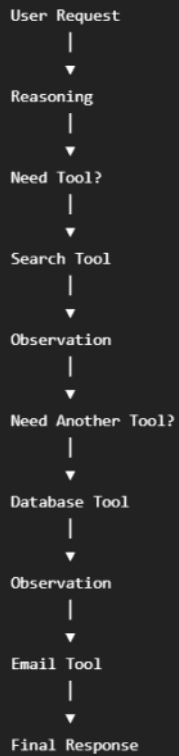
8. Final Response

The LLM converts structured data into a natural-language answer.

"The current temperature in Bangalore is 24°C with a 70% chance of rain. Carry an umbrella."

Multiple Tool Calling Architecture

For more complex tasks, the agent can call multiple tools.



Example:

"Find the cheapest flight and email it."

Tools:

- Flight Search
- Price Comparator
- Email Service

Planning & Reasoning

Planning and Reasoning System are AI agent architecture that enable an LLM to **break down complex goals, decide on a sequence of actions**, and adapt based on intermediate results.

These system are the foundation of modern AI agents.

Common Planning & Reasoning Patterns

1. ReAct (Reason + Act)

Idea: Think → Act → Observe → Repeat.

```
graph TD
    Thought --> Action
    Action --> Observation
    Observation --> Thought
    Thought --> FinalAnswer[Final Answer]
```

Best for:

- Dynamic tasks
- Tool use
- Search-based workflows

Example:

- Search the web
- Read results
- Search again if needed
- Answer

2. Plan-and-Execute

Idea: Create the complete plan first, then execute it.

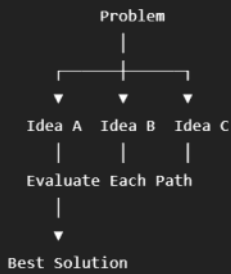
```
graph TD
    Goal --> Planner
    Planner --> TaskList[Task List]
    TaskList --> Executor
    Executor --> FinalAnswer[Final Answer]
```

Best for:

- Travel planning
- Report generation
- Software development
- Business workflows

3. Tree of Thoughts (ToT)

Instead of following a single reasoning path, the model explores **multiple possible approaches** and selects the best one.



Best for:

- Complex reasoning
- Optimization
- Puzzle solving

4. Chain of Thought (CoT)

The model reasons **step by step** before answering.

Example:

Question:
John has 5 apples...

The model performs intermediate reasoning before giving the answer.

Best for:

- Mathematics
- Logical reasoning
- Multi-step problem solving

5. Self-Reflection / Self-Correction

The agent reviews its own output before responding.

Generate Answer



Review Answer



Correct Mistakes



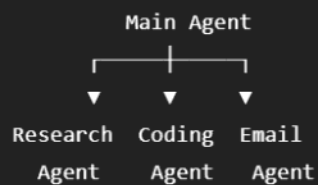
Final Answer

Useful for:

- Code generation
- Document review
- High-accuracy tasks

6. Multi-Agent Planning

One orchestrator delegates work to specialist agents.



Best for:

- Enterprise workflows
- Research automation
- Complex business processes

Comparison

Pattern	Best Use Case	Strength
Chain of Thought	Step-by-step reasoning	Logical problem solving
ReAct	Tool usage	Dynamic decision making
Plan-and-Execute	Structured workflows	Efficient execution
Tree of Thoughts	Multiple solution paths	Better optimization
Self-Reflection	Quality improvement	Error reduction
Multi-Agent	Large, complex tasks	Scalability and specialization

How to Choose

- Need external tools? → ReAct
- Task can be decomposed into clear steps? → Plan-and-Execute
- Need to compare multiple possible solutions? → Tree of Thoughts
- Need mathematical or logical reasoning? → Chain of Thought
- Need high reliability? → Self-Reflection
- Need domain specialists? → Multi-Agent

Skills and Instruction Files are important concepts

1. What are Skills?

A **skill** is a reusable capability that an AI agent can invoke to perform a specific type of task.

Think of a skill as a **specialized module** that combines:

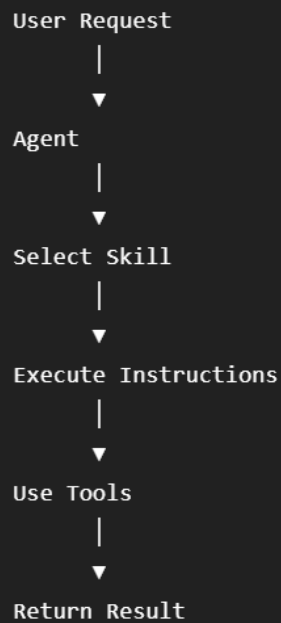
- Instructions
- Tools
- Prompt templates
- Business logic

Instead of writing the same prompts repeatedly, you package them into reusable skills.

Example Skills

- Email Writing
- SQL Query Generator
- Resume Analyzer
- Code Reviewer
- RAG Search
- Calendar Scheduler

Skill Architecture



2. What is an Instruction File?

An **instruction file** contains the rules and behavior that define how an AI agent or skill should operate.

It is similar to a **system prompt**, but organized in a maintainable, version-controlled format.

Example Instruction File

You are an HR assistant.

Responsibilities:

- Answer HR questions.
- Use the HR document search tool.
- Never guess.
- If information is unavailable, say so.
- Always cite the policy document.

Instead of embedding this prompt in code, it is stored separately.

Best Practices

Skills

- Keep them focused on one responsibility.
- Make them reusable across projects.
- Define clear inputs and outputs.
- Combine multiple tools when needed.

Instruction Files

- Clearly define the agent's role.
- Specify when to use each tool.
- Define output formats.
- Include examples for complex tasks.
- Specify what the agent should do when it lacks enough information.
- Keep instructions modular and version-controlled.

Token optimization and context management

What is Token Optimization?

Token optimization is the practice of **using the fewest tokens necessary** while preserving the quality of the response.

Why it matters:

- Lower API cost
- Faster responses
- Reduced latency
- Fits within the model's context window

What is Context Management?

Context management is the process of deciding **what information should be included in the prompt** sent to the LLM.

Not everything needs to be included—only the most relevant information.

Techniques for Token Optimization

1. Prompt Compression

Instead of lengthy instructions:



You are an AI assistant that always...
(500 words)

Use concise, clear instructions:



Answer using only the provided context.
If unsure, say you don't know.

2. RAG Retrieval (Top-K)

Retrieve only the most relevant document chunks.

Example:

User Query



Top 3 Chunks



LLM

Instead of sending 50 documents, send only the most relevant 3–5 chunks.

3. Semantic Chunking

Split documents by meaning instead of arbitrary character counts.

Benefits:

- Better retrieval quality
- Fewer irrelevant tokens

4. Conversation Summarization

Long conversations consume many tokens.

Instead of sending:

100 previous messages

Send:

Conversation Summary

+

Recent Messages

This preserves context while reducing token usage.

5. Sliding Context Window

Keep only the most recent and relevant messages.

Old Messages

↓

Discard

Recent Messages

↓

Keep

Useful for chatbots and long-running agents.

6. Remove Duplicate Context

Avoid sending the same information repeatedly.

Bad:

- Same document repeated
- Same instruction repeated

Good:

Include each piece of information once.

7. Structured Outputs

Instead of verbose text:

```
The customer name is John.
```

Use:

```
</> JSON
{
  "name": "John"
}
```

Structured outputs are typically shorter and easier for downstream systems to process.

8. Cache Static Prompts

Examples:

- System prompts
- Company policies
- Frequently used instructions

Many serving systems can reuse these, reducing repeated computation.

9. Context Filtering

Only include information relevant to the user's question.

Example:

User:

What's the refund policy?

Include:

- Refund policy

Exclude:

- Leave policy
- Travel policy
- Payroll policy

10. Hybrid Search

Combine:

- Keyword search (BM25)
- Semantic search (embeddings)

This often retrieves fewer but more relevant documents, improving both quality and token efficiency.

Best Practices

- Keep prompts concise and focused.
- Retrieve only relevant context (Top-K).
- Summarize older conversation history.
- Remove duplicate or irrelevant content.
- Use structured outputs when possible.
- Monitor token usage and costs.
- Cache reusable prompts or common prefixes when supported.

Multi-agent Orchestration

Multi-Agent Orchestration is an architecture where multiple specialized AI agents collaborate under a coordinate (orchestrator) to solve complex tasks.

Why Multi-Agent Orchestration?

A single agent may struggle with large, complex workflows.

For example:

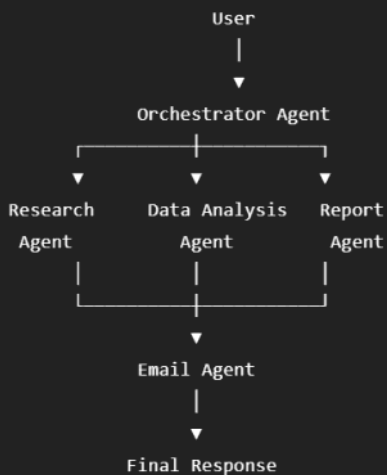
"Analyze last quarter's sales, identify trends, prepare a PowerPoint, and email it to the leadership team."

This task involves:

- Data analysis
- Visualization
- Report writing
- Emailing

Rather than one agent handling everything, specialized agents perform each task.

Architecture



Workflow

User Request

"Research AI market trends, summarize them, and email the report."

Step 1 – Orchestrator

Understands the request and breaks it into subtasks.

Step 2 – Assign Tasks

- **Research Agent** → Collects market data
- **Summarization Agent** → Creates the report
- **Email Agent** → Sends or drafts the email

Step 3 – Aggregate Results

The orchestrator combines the outputs.

Step 4 – Final Response

Returns the completed task to the user.

Components

1. Orchestrator

Responsibilities:

- Understand the goal
- Create task plan
- Select agents
- Monitor execution
- Handle failures
- Combine outputs

2. Specialist Agents

Examples:

- Research Agent
- SQL Agent
- Coding Agent
- RAG Agent
- Finance Agent
- Email Agent
- Calendar Agent

Each has its own:

- Instructions
- Tools
- Memory
- Expertise

3. Communication

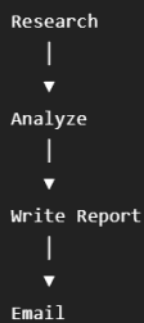
Agents exchange:

- Task descriptions
- Intermediate results
- Status updates
- Errors

Orchestration Patterns

1. Sequential

Each agent depends on the previous one.



Best for: Pipelines with clear dependencies.

2. Parallel

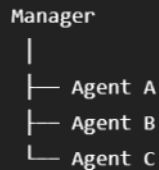
Multiple agents work simultaneously.



Best for: Independent subtasks, improving speed.

3. Hierarchical

One manager oversees multiple worker agents.



Best for: Large enterprise workflows.

Advantages

- Modular and easier to maintain.
- Specialists improve task quality.
- Parallel execution reduces latency.
- Easy to add new capabilities by introducing new agents.
- Better scalability for enterprise systems.

Challenges

- Increased coordination complexity.
- Passing the right context between agents.
- Higher infrastructure and API costs.
- Conflict resolution when agents disagree.
- Monitoring and debugging distributed workflows.

Single Agent vs Multi-Agent

Single Agent

One agent performs all tasks

Simpler architecture

Limited specialization

Easier to build

Harder to scale

Multi-Agent

Multiple specialized agents collaborate

More scalable and modular

Domain-specific expertise

Better for complex enterprise workflows

Easier to extend with new agents